ToDOO Project Explained

Frontend, backend, JWT authentication, and how everything connects

Created for C:/Users/aliib/Desktop/ToDOO on 2026-06-24

Simple mental model

The browser shows the React app. The Express server handles authentication. A JWT is the signed proof that the browser sends back when it wants to prove who is logged in.

```
Browser / React app
  - Shows login, register, and todo screens
  - Stores jwt_token in localStorage
  - Stores todos:userId in localStorage

          calls /api/auth/...
                 |
                 v
Vite dev server proxy
  - Serves the React app
  - Forwards /api requests to the backend

                 |
                 v
Express backend on port 4000
  - Registers users
  - Logs users in
  - Hashes passwords with bcryptjs
  - Signs and verifies JWTs
  - Stores demo users in server/data/users.json
```

1. Project in one minute

ToDOO is a todo list app with authentication. A user can register, log in, add todos, mark todos as done, delete todos, refresh the page, and stay logged in while the token is valid.

There are two main sides:

- Frontend: the React app inside src/. It draws the screens and reacts to clicks and form submissions.
- Backend: the Express API inside server/. It creates accounts, checks passwords, creates JWTs, and verifies JWTs.

The most important thing to remember is this: authentication is handled by the backend, but todos are still saved in the browser. The backend stores users only. The todo list is stored in localStorage under a key that includes the user id.

In plain English:

- The frontend asks: Can I register or log in?
- The backend answers: Yes, here is a signed token.
- The frontend saves that token.
- Later, the frontend sends the token back.
- The backend checks the token and says who the user is.

2. Folder and file map

This is the part of the project you actually need to understand. The node_modules folder is installed packages, and dist is build output, so they are not where the main logic lives.

```
ToDOO/
  package.json              Scripts and dependencies
  vite.config.js            Vite setup and /api proxy
  README.md                 Short project notes
  .gitignore                Keeps local data and dependencies out of commits

  server/
    index.js                Express API, JWT auth, user storage
    data/users.json         Created locally when users register

  src/
    main.jsx                Starts React and renders <App />
    index.css               Global page reset and body layout
    App.jsx                 Main app logic: auth state + todo state
    App.css                 Styles for auth card, todo card, forms
    api/auth.js             Frontend helper for auth HTTP requests
    components/
      AuthForm.jsx          Login/register form
      TodoItem.jsx          One todo row
      TodoItem.css          Styles for one todo row
```

What each generated folder means:

- node_modules/: packages installed by npm. You almost never edit this.
- dist/: production frontend build created by npm run build. You usually do not edit this by hand.
- server/data/: local demo data. This can contain password hashes, so it should stay out of git.

3. package.json explained

package.json tells npm what this project is, what packages it uses, and which commands are available.

Important scripts:

- npm run dev: starts the backend and frontend at the same time.
- npm run server: starts the Express API with node server/index.js.
- npm run client: starts Vite, which serves the React app.
- npm run build: creates a production frontend build in dist/.
- npm run preview: previews the production build.

The dev command uses concurrently:

```
"dev": "concurrently \"npm run server\" \"npm run client\""
```

That means one terminal command can run both parts of the app. Without concurrently, you would need two terminals: one for the backend and one for the frontend.

Dependencies:

- react: builds the frontend UI.
- react-dom: places React into the browser page.
- react-hook-form: helps with form validation and form submission.
- express: creates the backend API server.
- cors: allows browser requests from allowed origins.
- jsonwebtoken: creates and verifies JWT tokens.
- bcryptjs: hashes passwords and checks login passwords.

Dev dependencies:

- vite: frontend development server and build tool.
- @vitejs/plugin-react: lets Vite understand React.
- concurrently: runs frontend and backend commands together.

4. How the app starts

When you run:

```
npm run dev
```

this happens:

```
1. concurrently starts two commands.
2. npm run server starts Express on http://localhost:4000.
3. npm run client starts Vite, usually on http://localhost:5173.
4. The browser opens the React app from Vite.
5. When React calls /api/auth/..., Vite forwards it to Express.
```

The default frontend URL is usually http://localhost:5173. If that port is already busy, Vite can use another port. The backend default URL is http://localhost:4000.

5. vite.config.js explained

vite.config.js configures the frontend dev server.

```
export default defineConfig({
  plugins: [react()],
  server: {
    proxy: {
      '/api': 'http://localhost:4000',
    },
  },
})
```

The important part is the proxy.

The React code calls a relative URL like:

```
/api/auth/login
```

The Vite proxy forwards that to:

```
http://localhost:4000/api/auth/login
```

This keeps frontend code clean. The frontend does not need to know the full backend URL everywhere. It just calls /api/... and lets Vite forward the request.

6. main.jsx explained

main.jsx is the frontend entry point. It starts React.

```
createRoot(document.getElementById('root')).render(
  <StrictMode>
    <App />
  </StrictMode>,
)
```

What this means:

- document.getElementById('root') finds the root div in index.html.
- createRoot(...) tells React: this is where the app lives.
- <App /> is the main component for the whole app.
- StrictMode helps React warn about common development mistakes.

main.jsx does not contain todo logic or auth logic. It only starts the app.

7. index.css explained

index.css contains global styles for the whole page.

It does three simple things:

- Resets margin and padding for all elements.
- Uses border-box sizing so width calculations are easier.
- Styles the body with a font, light gray background, and centered layout.

This is why the app appears as a centered card on a light background.

8. App.jsx is the main brain

App.jsx controls the big decisions:

- Is the user logged in?
- Are we still checking a saved token?
- Which todos should be shown?
- Should the user see AuthForm or the todo list?

Important state:

```
user
  The logged-in user object.
  null means nobody is logged in.

isCheckingAuth
  true while the app checks whether an existing token is still valid.

todos
  The todo items currently shown on screen.

activeTodoUserId
  A safety value that says which user the current todo list belongs to.
```

The first important effect checks login on refresh:

```
useEffect(() => {
  if (!getToken()) return

  getCurrentUser()
    .then(data => setUser(data.user))
    .catch(() => removeToken())
    .finally(() => setIsCheckingAuth(false))
}, [])
```

Easy explanation:

- If there is no token, do nothing.
- If there is a token, ask the backend who this token belongs to.
- If the backend accepts it, save the user in React state.
- If the backend rejects it, delete the token.
- When the check is done, stop showing the loading state.

This is what makes refresh login work.

9. How App.jsx stores todos

Todos are saved by user id:

```
const todosKey = userId => `todos:${userId}`
```

So if a user has id abc123, the browser key becomes:

```
todos:abc123
```

This matters because two users can use the same browser without sharing the same todo list.

When user changes, App loads that user's todos:

```
if (user) {
  setTodos(loadTodos(user.id))
  setActiveTodoUserId(user.id)
} else {
  setTodos([])
  setActiveTodoUserId(null)
}
```

When todos change, App saves them:

```
if (user && activeTodoUserId === user.id) {
  localStorage.setItem(todosKey(user.id), JSON.stringify(todos))
}
```

The activeTodoUserId check prevents the app from accidentally saving an empty todo list over the wrong user's saved todos during login refresh timing.

10. App.jsx todo functions

Adding a todo:

```
function onSubmit({ title }) {
  setTodos(prev => [...prev, { id: crypto.randomUUID(), title, done: false }])
  reset()
}
```

Easy explanation:

- Take the title from the form.
- Create a new todo object.
- Give it a unique id.
- Set done to false.
- Add it to the end of the old todos array.
- Reset the input field.

Toggling a todo:

```
function toggleTodo(id) {
  setTodos(prev => prev.map(t => t.id === id ? { ...t, done: !t.done } : t))
}
```

Easy explanation:

- Look through every todo.
- If the id matches, flip done from false to true or true to false.
- If it does not match, leave the todo unchanged.

Deleting a todo:

```
function deleteTodo(id) {
  setTodos(prev => prev.filter(t => t.id !== id))
}
```

Easy explanation:

- Keep every todo except the one with the matching id.

Logging out:

```
function logout() {
  removeToken()
  setUser(null)
}
```

Easy explanation:

- Delete the token.
- Clear the user state.
- Because user is now null, App shows the login/register form again.

11. AuthForm.jsx explained

AuthForm is the screen shown when nobody is logged in.

It has two modes:

- login
- register

State inside AuthForm:

```
mode
  Controls whether the form is login or register.

serverError
  Shows backend errors, such as invalid password or email already registered.

isSubmitting
  Shows a loading state and disables the button while the request is running.
```

AuthForm uses react-hook-form:

```
const {
  register,
  handleSubmit,
  reset,
  formState: { errors },
} = useForm()
```

Easy explanation:

- register connects inputs to the form system.
- handleSubmit runs validation before calling your submit function.
- reset clears the form.
- errors contains validation messages.

For register mode, the form shows name, email, and password. For login mode, it shows email and password only.

12. AuthForm submit flow

When the user submits AuthForm:

```
1. Clear old server errors.
2. Set isSubmitting to true.
3. If mode is register, call registerUser(values).
4. If mode is login, call loginUser({ email, password }).
5. Backend returns token and user.
6. saveToken(data.token) stores the JWT in localStorage.
7. onAuth(data.user) tells App the user is logged in.
8. reset() clears the form.
9. If anything fails, show error.message.
10. Set isSubmitting back to false.
```

The key line is:

```
onAuth(data.user)
```

AuthForm does not own the whole app. App owns the user state. So AuthForm calls onAuth to tell App: login worked, here is the user.

13. src/api/auth.js explained

This file is the frontend's auth helper. It keeps localStorage and fetch logic in one place.

Functions:

```
getToken()
  Reads jwt_token from localStorage.

saveToken(token)
  Saves jwt_token in localStorage.

removeToken()
  Deletes jwt_token from localStorage.

authRequest(path, options)
  Calls fetch, sends JSON headers, adds Authorization if a token exists,
  parses JSON, and throws readable errors.

registerUser(values)
  POST /api/auth/register

loginUser(values)
  POST /api/auth/login

getCurrentUser()
  GET /api/auth/me
```

The most important part:

```
headers: {
  'Content-Type': 'application/json',
  ...(token ? { Authorization: `Bearer ${token}` } : {}),
  ...options.headers,
}
```

Easy explanation:

- Tell the backend we are sending JSON.
- If a token exists, attach it as Authorization: Bearer token.
- Add any extra headers passed by the caller.

Bearer means: I am carrying this token as my proof.

14. TodoItem.jsx explained

TodoItem renders one todo row. It does not own the todo list.

Props:

```
todo
  The item to display: id, title, done.

onToggle
  Function from App. Called when the checkbox changes.

onDelete
  Function from App. Called when the delete button is clicked.
```

Why this is good React style:

- App owns the todos array.
- TodoItem only displays one item.
- TodoItem reports user actions back to App.

That means the data has one owner, which makes the app easier to understand.

15. CSS files explained

App.css styles the main card, auth form, todo form, buttons, errors, and responsive layout.

Important classes:

- .card: white container around the UI.
- .card__header: top row with title and sign-out button.
- .form__input: text input styling.
- .form__input--error: red border when validation fails.
- .form__btn: main black submit button.
- .auth-tabs: login/register segmented buttons.
- .auth-tabs__btn--active: selected tab style.
- .empty: message when there are no todos.
- .list: vertical todo list.

TodoItem.css styles each todo row:

- .item: horizontal row layout.
- .item__check: checkbox.
- .item__title: todo text.
- .item__title--done: line-through when complete.
- .item__delete: delete button.

16. Backend overview

The backend lives in server/index.js. It is responsible for users and JWTs.

It does these jobs:

- Starts an Express server.
- Accepts JSON request bodies.
- Allows frontend requests with CORS.
- Reads users from server/data/users.json.
- Saves users to server/data/users.json.
- Hashes passwords.
- Compares login passwords.
- Creates JWT tokens.
- Verifies JWT tokens.
- Provides auth routes.

The backend does not currently save todos. That would be a future feature.

17. Backend imports explained

```
import bcrypt from 'bcryptjs'
import cors from 'cors'
import express from 'express'
import jwt from 'jsonwebtoken'
import { randomUUID } from 'node:crypto'
import { mkdir, readFile, writeFile } from 'node:fs/promises'
import path from 'node:path'
import { fileURLToPath } from 'node:url'
```

What each import does:

- bcryptjs: protects passwords by hashing them.
- cors: lets browser apps call the API from allowed frontend origins.
- express: creates the backend server and routes.
- jsonwebtoken: signs and verifies JWT tokens.
- randomUUID: creates unique user ids.
- fs/promises: reads and writes files with async/await.
- path and fileURLToPath: build safe file paths for users.json.

18. Backend constants explained

```
const app = express()
const PORT = process.env.PORT || 4000
const JWT_SECRET = process.env.JWT_SECRET || 'change-this-secret-in-production'
const TOKEN_EXPIRES_IN = '1h'
```

Easy explanation:

- app is the Express server.
- PORT is where the backend listens. Default: 4000.
- JWT_SECRET is the private signing key for tokens.
- TOKEN_EXPIRES_IN makes each token last 1 hour.

Important: change-this-secret-in-production is only okay for learning. A real deployed app should set JWT_SECRET to a strong random value in the environment.

The server also defines:

```
const DATA_DIR = path.join(__dirname, 'data')
const USERS_FILE = path.join(DATA_DIR, 'users.json')
```

That means user data is stored in:

```
server/data/users.json
```

19. Backend middleware explained

Middleware is code that runs before route handlers.

The app uses:

```
app.use(cors({ origin: 'http://localhost:5173' }))
app.use(express.json())
```

cors allows requests from the frontend origin. express.json reads JSON request bodies so routes can use req.body.

The custom auth middleware is requireAuth:

```
function requireAuth(req, res, next) {
  const authHeader = req.headers.authorization
  const token = authHeader?.startsWith('Bearer ') ? authHeader.slice(7) : null

  if (!token) {
    return res.status(401).json({ message: 'Missing token' })
  }

  try {
    req.user = jwt.verify(token, JWT_SECRET)
    next()
  } catch {
    res.status(401).json({ message: 'Invalid or expired token' })
  }
}
```

Easy explanation:

- Look for the Authorization header.
- Make sure it starts with Bearer.
- Remove the word Bearer and keep only the token.
- If no token exists, reject the request.
- If a token exists, verify it with JWT_SECRET.
- If it is valid, store the decoded user on req.user and continue.
- If it is invalid or expired, reject the request.

20. Backend helper functions explained

readUsers():

- Tries to read users.json.
- Parses the file as JSON.
- If the file does not exist yet, returns [].

saveUsers(users):

- Creates server/data if needed.
- Writes the users array to users.json.
- Uses pretty JSON with indentation, so it is readable.

publicUser(user):

- Returns only id, name, and email.
- Does not return passwordHash.
- This protects private backend data from being sent to the browser.

createToken(user):

- Takes the safe user data.
- Signs it with JWT_SECRET.
- Adds an expiry of 1 hour.
- Returns the JWT string.

21. Register route explained

Route:

```
POST /api/auth/register
```

Frontend function:

```
registerUser(values)
```

Step by step:

```
1. Read name, email, and password from req.body.
2. Trim name and email.
3. Lowercase the email.
4. Check that name, email, and password exist.
5. Check that password has at least 6 characters.
6. Read existing users.
7. Check if the email is already registered.
8. Hash the password with bcrypt.hash(password, 10).
9. Create a user with id, name, email, and passwordHash.
10. Save the user.
11. Create a JWT.
12. Return { token, user: publicUser(user) }.
```

The route returns status 201 because a new user was created.

Why hash the password:

- The backend should never store plain text passwords.
- bcrypt turns the password into a hash.
- On login, bcrypt.compare can check the typed password against the hash.

22. Login route explained

Route:

```
POST /api/auth/login
```

Frontend function:

```
loginUser(values)
```

Step by step:

```
1. Read email and password from req.body.
2. Lowercase the email.
3. Check that both values exist.
4. Read existing users.
5. Find the user by email.
6. Use bcrypt.compare(password, user.passwordHash).
7. If no user or password does not match, return 401.
8. If login is correct, create a JWT.
9. Return { token, user: publicUser(user) }.
```

Why the error says Invalid email or password:

- It does not reveal whether the email exists.
- This is a small security habit.

23. /me route explained

Route:

```
GET /api/auth/me
```

Frontend function:

```
getCurrentUser()
```

This route is protected:

```
app.get('/api/auth/me', requireAuth, async (req, res) => {
  ...
})
```

That means requireAuth runs first.

Step by step:

```
1. Frontend sends Authorization: Bearer <token>.
2. requireAuth verifies the token.
3. If the token is bad, the backend returns 401.
4. If the token is good, req.user contains the decoded token data.
5. The route finds the full user by req.user.id.
6. If the user exists, return { user: publicUser(user) }.
```

Why /me is needed:

- Refreshing the page clears React state.
- localStorage still has the token.
- /me lets React turn that token back into a user object.

24. JWT explained very simply

JWT means JSON Web Token.

In this project, it is a signed login proof.

It has three parts:

```
header.payload.signature
```

Header:

- Says what kind of token it is.
- Says what signing algorithm is used.

Payload:

- Contains safe user data from publicUser.
- Contains expiry information.

Signature:

- Created using JWT_SECRET.
- Proves the token was created by the server.
- Proves the token was not changed.

Very important: JWT is signed, not encrypted.

That means:

- The backend can detect changes to the token.
- The payload should not contain secrets.
- Do not put passwords in a JWT.

25. How frontend and backend connect

The frontend and backend are separate programs.

They connect using HTTP requests:

```
React component
  calls
src/api/auth.js
  calls fetch('/api/auth/login')
    goes through Vite proxy
      reaches Express route /api/auth/login
        returns JSON
      Vite sends JSON back
  auth.js returns data
React updates state
```

The frontend does not import backend functions. It sends requests to backend URLs.

The backend does not import React components. It receives HTTP requests and sends JSON responses.

This separation is important. It means the frontend cares about screens and user interaction. The backend cares about data, security, and API responses.

26. Register flow from click to token

Full register flow:

```
1. User opens the app.
2. App sees user is null.
3. App shows AuthForm.
4. User clicks Register.
5. AuthForm sets mode to register.
6. User enters name, email, and password.
7. react-hook-form validates the inputs.
8. AuthForm calls registerUser(values).
9. registerUser sends POST /api/auth/register.
10. Vite proxies the request to Express.
11. Express validates the request.
12. Express hashes the password.
13. Express saves the user.
14. Express signs a JWT.
15. Express returns token and public user.
16. AuthForm saves the token.
17. AuthForm calls onAuth(data.user).
18. App sets user.
19. App loads todos for that user.
20. App shows the todo screen.
```

Example request body:

```
{
  "name": "Ali",
  "email": "ali@example.com",
  "password": "secret123"
}
```

Example response shape:

```
{
  "token": "eyJhbGciOi...",
  "user": {
    "id": "generated-user-id",
    "name": "Ali",
    "email": "ali@example.com"
  }
}
```

27. Login flow from click to todo screen

Full login flow:

```
1. User enters email and password.
2. AuthForm validates the form.
3. AuthForm calls loginUser({ email, password }).
4. loginUser sends POST /api/auth/login.
5. Backend reads users.json.
6. Backend finds the user by email.
7. Backend uses bcrypt.compare.
8. If wrong, backend returns 401.
9. If correct, backend signs a JWT.
10. Backend returns token and public user.
11. Frontend saves jwt_token.
12. App receives the user.
13. App loads todos for that user id.
14. Todo screen appears.
```

Wrong password flow:

```
backend returns 401
authRequest throws an Error
AuthForm catches the Error
serverError is shown under the form
```

28. Refresh flow

Refreshing the browser clears React memory, but it does not clear localStorage.

That is why the app can stay logged in.

Refresh flow:

```
1. Page reloads.
2. main.jsx starts React again.
3. App starts with user = null.
4. App checks localStorage for jwt_token.
5. If token exists, App shows "Checking your login...".
6. App calls getCurrentUser().
7. getCurrentUser sends GET /api/auth/me.
8. authRequest attaches Authorization: Bearer <token>.
9. Backend verifies the token.
10. If token is valid, backend returns the user.
11. App sets user and shows todo screen.
12. If token is invalid or expired, App removes it and shows AuthForm.
```

This is one of the most important flows in the project.

29. Logout flow

Logout is simple because JWT auth is stateless in this project.

Flow:

```
1. User clicks Sign out.
2. App calls removeToken().
3. removeToken deletes jwt_token from localStorage.
4. App sets user to null.
5. App clears current todos from state.
6. App shows AuthForm again.
```

The backend does not need a logout route in this simple version because the frontend forgets the token.

In a more advanced app, you might add refresh tokens, token blacklists, or httpOnly cookies. This project keeps it beginner-friendly.

30. Todo flow

Todos are frontend-only right now.

Add todo:

```
1. User types title.
2. react-hook-form validates title.
3. App creates { id, title, done: false }.
4. App adds it to todos state.
5. React redraws the list.
6. useEffect saves todos to localStorage.
```

Toggle todo:

```
1. User clicks checkbox.
2. TodoItem calls onToggle(todo.id).
3. App maps over todos.
4. Matching todo flips done.
5. React redraws the item with line-through style.
6. useEffect saves todos to localStorage.
```

Delete todo:

```
1. User clicks x.
2. TodoItem calls onDelete(todo.id).
3. App filters out that todo.
4. React redraws the list.
5. useEffect saves todos to localStorage.
```

31. Important concepts in beginner words

Component:

- A piece of UI.
- App, AuthForm, and TodoItem are components.

State:

- Data React remembers.
- user, todos, mode, and serverError are state.

Props:

- Values passed from a parent component to a child component.
- App passes todo, onToggle, and onDelete to TodoItem.

Effect:

- Code that runs after rendering.
- App uses effects to check login and save todos.

API route:

- A backend URL that does a job.
- /api/auth/login is an API route.

Middleware:

- Code that runs before a route.
- requireAuth checks the token before /me runs.

Hash:

- A protected version of a password.
- The backend stores passwordHash, not the original password.

JWT:

- A signed text token that proves the user logged in.

Proxy:

- A dev-server helper that forwards /api requests to the backend.

32. What to inspect while learning

Browser DevTools localStorage:

- jwt_token
- todos:<userId>

Network tab:

- POST /api/auth/register
- POST /api/auth/login
- GET /api/auth/me

Backend data:

- server/data/users.json
- You should see users with id, name, email, and passwordHash.
- You should not see plain text passwords.

Terminal:

- The backend should log that it is running on port 4000.
- If requests fail, backend errors usually show there.

33. Common problems and fixes

Problem: I registered, but login says invalid email or password.

- Check that the email is typed the same way.
- Check that the password is correct.
- Remember that password must be at least 6 characters.

Problem: The frontend cannot call /api.

- Make sure npm run server is running.
- Make sure the API is on port 4000.
- Make sure vite.config.js still has the /api proxy.

Problem: Refresh logs me out.

- The token may have expired after 1 hour.
- The token may be missing from localStorage.
- JWT_SECRET may have changed since the token was created.

Problem: Direct API calls from another frontend port fail.

- Use the Vite proxy.
- Or update cors({ origin: ... }) in server/index.js.

34. Security notes

This project is good for learning. For production, improve these parts:

- Use a strong JWT_SECRET environment variable.
- Do not rely on the fallback secret.
- Use a real database instead of users.json.
- Consider httpOnly cookies for tokens in higher-security apps.
- Add rate limiting to login/register.
- Add stronger validation on the backend.
- Use HTTPS in production.
- Move todos to backend routes if you want multi-device sync.

The current password handling is already much better than plain text because passwords are hashed with bcryptjs.

Still, server/data/users.json should stay out of git because password hashes are sensitive.

35. Best next step

The clearest next feature is backend todo storage.

You would add routes like:

```
GET    /api/todos
POST   /api/todos
PATCH  /api/todos/:id
DELETE /api/todos/:id
```

Each route would use requireAuth.

That means:

- The backend would know which user is making the request.
- Todos could be saved by user id.
- The same user could log in from another browser and see the same todos.

36. Final recap

If you understand these four sentences, you understand the project:

- React shows the screens and stores UI state.
- AuthForm sends login/register data through src/api/auth.js.
- Express checks users, hashes passwords, and signs/verifies JWTs.
- App uses the token to keep the user logged in and uses localStorage for that user's todos.

One last tiny summary:

```
AuthForm -> auth.js -> Vite proxy -> Express route
Express route -> token + user -> auth.js -> App state
App state -> todo screen
todo screen -> localStorage
refresh -> token -> /me -> user restored
```